

Robotic OT Tracker

PowerApps Conversion Guide

A complete step-by-step guide for converting the Robotic OT Tracker
from Google Sheets to Microsoft PowerApps + SharePoint + Power Automate

Standard M365 Corporate Account (SharePoint only, no Dataverse)

CGH da Vinci Programme | Robotic Surgery

Document Version 1.0 | 01 Sep 2026

Contents

Chapter 1 Introduction & Prerequisites

- 1.1 Purpose of this guide
- 1.2 What you will need before starting
- 1.3 Time estimate
- 1.4 Glossary of terms

Chapter 2 Architecture Overview

- 2.1 Current system (Google Sheets + GAS + GitHub Pages)
- 2.2 New system (SharePoint + Power Automate + Canvas App)
- 2.3 Why Power Automate flows for all writes

Chapter 3 SharePoint Lists Setup

- 3.1 Create the SharePoint site
- 3.2 Instruments list — all columns
- 3.3 Consumables list — all columns
- 3.4 Faults list — all columns
- 3.5 AuditTrail list — all columns
- 3.6 Index the key columns for performance

Chapter 4 Power Automate Flows

- 4.1 Overview — how flows connect to Canvas App
- 4.2 Flow: Log Instrument Use
- 4.3 Flow: Log Consumable Use
- 4.4 Flow: Log Fault
- 4.5 Flow: Undo Use
- 4.6 Flow: Change Instrument Status
- 4.7 Flow: Condemn Instrument
- 4.8 Flow: Add Instrument
- 4.9 Flow: Add Consumable

Chapter 5 Build the Canvas App

- 5.1 Create the app and connect SharePoint data
- 5.2 App.OnStart — load data into collections
- 5.3 Screen: Dashboard
- 5.4 Screen: Instruments
- 5.5 Screen: Consumables
- 5.6 Screen: Log Use
- 5.7 Screen: Manage (Change Status & Condemn)
- 5.8 Screen: Faults
- 5.9 Screen: Audit Trail
- 5.10 Navigation — connecting all screens

Chapter 6 Data Migration from Google Sheets

- 6.1 Export instruments and consumables from Google Sheets
- 6.2 Format the spreadsheet for SharePoint import
- 6.3 Import to SharePoint lists
- 6.4 Verify the imported data

Chapter 7 Testing Checklist

Chapter 8 Troubleshooting

Chapter A Appendix A: SharePoint Column Quick Reference

Chapter B Appendix B: Key Power Fx Formulas

Chapter 1 — Introduction & Prerequisites

1.1 Purpose of this guide

This guide walks you through converting the **Robotic OT Tracker** — currently running as a web app backed by Google Sheets — into a fully functional **Microsoft PowerApps Canvas App** that stores all data in **SharePoint Online** and processes writes through **Power Automate** flows.

The guide assumes you have **no prior experience** with PowerApps, Power Automate, or SharePoint administration. Every step is written as a numbered instruction. You can follow it top to bottom without skipping any section.

1.2 What you will need before starting

Before you begin, confirm that you have access to all of the following:

Item	What it is	Where to get it
Microsoft 365 account	A standard corporate M365 account (E1, E3, or E5). No Dataverse or premium licence needed.	Your IT administrator
PowerApps access	Included in M365. Go to make.powerapps.com and log in with your M365 account.	make.powerapps.com
Power Automate access	Included in M365. Go to make.powerautomate.com .	make.powerautomate.com
SharePoint access	Included in M365. You need permission to create a new SharePoint site.	Your IT administrator (if you cannot create sites yourself)
The Google Sheet	The existing Robotic OT Tracker spreadsheet — for exporting current data in Chapter 6.	Current app owner
A modern web browser	Google Chrome or Microsoft Edge (latest version). Do not use Internet Explorer.	Your computer

■ *If you cannot create SharePoint sites yourself, raise a request with your IT administrator to create a team site called "Robotic OT Tracker" before continuing with Chapter 3.*

1.3 Time estimate

Plan for the following total effort:

Task	Estimated time
Chapter 3: Create SharePoint lists	1–2 hours
Chapter 4: Build Power Automate flows (8 flows)	4–6 hours
Chapter 5: Build the Canvas App	8–12 hours
Chapter 6: Data migration	1–2 hours
Chapter 7: Testing	1–2 hours
Total	15–24 hours

✓ *This is a significant project. Work through one chapter at a time and save your work frequently. You can stop and resume at any chapter boundary.*

1.4 Glossary of terms

The following terms are used throughout this guide:

Term	Meaning
Canvas App	A Microsoft PowerApps application where you drag-and-drop controls and write formulas. The final product that nurses/staff will use on their phone or tablet.
SharePoint List	A table stored in SharePoint Online. Works like a spreadsheet tab — it has rows (items) and columns (fields). Replaces Google Sheets in the new system.
Power Automate Flow	An automated workflow that runs in the cloud. Used here to write data to two SharePoint lists atomically (primary record + audit trail) so neither can succeed without the other being attempted.
Power Fx	The formula language used in Canvas Apps. Similar to Excel formulas. Used to query data, validate input, and call flows.
Collection	A local in-memory table inside the Canvas App (like a cache). Data is loaded from SharePoint into a collection once at startup so the app feels fast.
Delegation	When the Canvas App sends a filter condition to SharePoint to run server-side (efficient). Non-delegable queries run on the device (only on the first 500 rows loaded). Always use delegable formulas for large lists.
PowerApps V2 trigger	The type of Power Automate trigger that lets a Canvas App call a flow and pass parameters to it. The standard method for connecting Canvas Apps to flows in M365.
Respond to a PowerApp	The final step in a Power Automate flow that sends a result back to the Canvas App. The Canvas App formula reads this result to determine what message to show the user.

Term	Meaning
Action	A single step inside a Power Automate flow. Examples: "SharePoint - Create item", "Initialize variable", "Condition".
Instrument	A reusable da Vinci robotic instrument (e.g. Maryland forceps). Has a fixed maximum use count.
Consumable	A single-use or limited-use item (e.g. clip applicator). Tracked by a balance count.
Serial Number (SN)	The unique identifier for each instrument or consumable. Stored in the "Title" column of the SharePoint list.

Chapter 2 — Architecture Overview

2.1 Current system

The current Robotic OT Tracker works as follows:

Layer	Technology	Role
UI	HTML + CSS + JavaScript (GitHub Pages)	What users see and click in their browser
Logic	Vanilla JavaScript (app.js)	All business rules, validation, state management
API	Google Apps Script (GAS) REST endpoint	Receives requests from the browser, reads/writes Google Sheets
Database	Google Sheets (4 tabs)	Instruments, Consumables, Faults, AuditTrail tabs

Data flow for a write operation (e.g. log instrument use):

- Browser calls GAS REST API: *saveConsumable*
- GAS writes to Google Sheets Instruments tab
- Browser then calls GAS REST API: *saveAudit*
- GAS writes to Google Sheets AuditTrail tab
- If step 4 fails silently, audit trail is missing — this is the key problem the new system solves.

2.2 New system

Layer	Technology	Role
UI	PowerApps Canvas App	What users see and interact with on phone, tablet, or browser
Logic	Power Fx formulas in Canvas App	Validation, local state, UI decisions
API / Automation	Power Automate Flows (8 flows)	Receives calls from Canvas App; writes to SharePoint atomically
Database	SharePoint Online (4 lists)	Instruments, Consumables, Faults, AuditTrail lists

Data flow for a write operation in the new system:

- Canvas App calls a Power Automate flow with all required parameters
- Flow writes to SharePoint primary list (e.g. Instruments)
- If primary write fails, flow returns "primary_failed" — nothing was changed

- If primary write succeeds, flow retries audit trail write up to 3 times (2 s apart)
- Flow returns "success", "audit_failed", or "primary_failed" to the Canvas App
- Canvas App shows the appropriate message to the user

2.3 Why Power Automate flows for all writes

In the original app, two separate API calls were made one after the other. If the network dropped between the two calls, the primary record was saved but no audit trail entry was created. This was a silent failure — the user saw no error and could not know the audit trail was incomplete.

In the new architecture, **every write operation is handled by a single Power Automate flow call**. The flow manages both the primary record write and the audit trail write. The Canvas App makes only *one* network call. The flow reports back exactly what happened.

■ **This is the most important architectural decision in this conversion. Do not skip building the flows and call SharePoint directly from the Canvas App — you will re-create the silent audit trail failure problem.**

Chapter 3 — SharePoint Lists Setup

3.1 Create the SharePoint site

You need a SharePoint site to host the four lists. Follow these steps:

1. Open a browser and go to **office.com**. Sign in with your M365 account.
2. Click the grid icon (■) in the top-left corner. Select **SharePoint**.
3. On the SharePoint home page, click **+ Create site** (top-left area).
4. Choose **Team site**.
5. Fill in the form:
 - Site name: **Robotic OT Tracker**
 - Description: CGH da Vinci Programme instrument tracking
 - Privacy: **Private** (only invited members can access)
 - Language: English
6. Click **Next**, then add any additional owners or members. Click **Finish**.
7. SharePoint will create the site. You will be taken to the new site. Copy the site URL from your browser address bar — you will need it in Chapter 4 and 5. It will look like:
https://yourorg.sharepoint.com/sites/RoboticOTTracker

✓ Write down your site URL now. You will enter it many times in Chapters 4 and 5.

3.2 Instruments list

The Instruments list stores every da Vinci robotic instrument.

Create the list

1. In your SharePoint site, click **New** in the top toolbar, then select **List**.
2. Choose **Blank list**.
3. Name: **Instruments**. Leave "Show in site navigation" checked. Click **Create**.

Rename the built-in Title column

4. The list starts with a column called "Title". Click the column header **Title**, then click **Column settings > Rename**.
5. Change the name to **SerialNumber**. Click **Save**.

■ SharePoint stores "SerialNumber" internally as "Title". In Power Automate, when you see a field called "Title" in the SharePoint action, that is your Serial Number field.

Add columns

Click **+** **Add column** (at the far right of the column headers) for each column below. Select the type shown, fill in the settings, then click **Save**.

Column name to enter	Type to select	Settings
InstrumentType	Choice	Add these choices (one per line): Monopolar scissors Fenestrated bipolar Cadiere forceps Maryland forceps Prograsp forceps Curved bipolar dissector Large needle driver Large suturecut needle driver Cautery hook Cautery spatula Fenestrated TIP-UP 0 degree lens 30 degree lens
Status	Choice	Add these choices: Circulation SB Instrument URO Set Non-sterile Complete Condemned Default value: Circulation
UsesLeft	Number	No decimal places. Leave min/max blank.
MaxLife	Number	No decimal places. Leave min/max blank.
LastUsed	Date and time	Format: Date only (no time).
LastCase	Single line of text	Leave all defaults.
Remarks	Multiple lines of text	Plain text. Leave all defaults.

■ Add columns in the exact order shown above with the exact names. Power Automate flows in Chapter 4 reference these names directly.

3.3 Consumables list

Create the list

1. Click **New > List > Blank list**.
2. Name: **Consumables**. Click **Create**.
3. Rename the "Title" column to **SerialNumber** (same as Section 3.2, steps 4–5).

Add columns

Column name	Type	Settings
ConsumableType	Choice	Add choices: Small clip applicator M/L clip applicator Large clip applicator Vessel sealer SureForm 60 SureForm 45 Harmonic scalpel
Balance	Number	No decimal places.
MaxBalance	Number	No decimal places.
Expiry	Date and time	Format: Date only.

Column name	Type	Settings
LastUsed	Date and time	Format: Date only.

3.4 Faults list

Create the list

1. Click **New > List > Blank list**. Name: **Faults**. Click **Create**.
2. Rename "Title" to **SerialNumber**.

Add columns

Column name	Type	Settings
FaultTimestamp	Date and time	Format: Date & Time.
InstrumentType	Single line of text	Leave defaults.
FaultType	Choice	Add choices: Wire broken Wire frail Cannot grasp Cannot close Jaw unable to open Intra-op failure Call back Cannot fire (stapler) Display issue (lens) Other
CaseNumber	Single line of text	Leave defaults.
Notes	Multiple lines of text	Plain text.
LoggedBy	Single line of text	Leave defaults.

3.5 AuditTrail list

This list records every action taken in the app — it is the permanent log.

Create the list

1. Click **New > List > Blank list**. Name: **AuditTrail**. Click **Create**.
2. Rename "Title" to **SerialNumber**.

Add columns

Column name	Type	Settings
EventTimestamp	Date and time	Format: Date & Time.
Event	Choice	Add choices: Use logged Consumable use logged Fault logged Use REVERSED Status changed Instrument condemned Instrument added Consumable added
InstrumentType	Single line of text	Leave defaults.
Detail	Multiple lines of text	Plain text. Stores the full event description.

Column name	Type	Settings
CaseNumber	Single line of text	Leave defaults.
LoggedBy	Single line of text	Leave defaults.

3.6 Index the key columns for performance

SharePoint can only filter efficiently on *indexed* columns. You must index SerialNumber on the Instruments, Consumables, Faults, and AuditTrail lists. Without this, the Canvas App will only load the first 500 rows correctly.

Repeat these steps for **each of the four lists**:

1. Open the list. Click the gear icon (⚙) in the top-right corner > **List settings**.
2. Scroll down to the **Columns** section. Click **Indexed columns**.
3. Click **Create a new index**.
4. For the "Primary column for this index", select **SerialNumber**. Leave "Secondary column" blank. Click **Create**.

✓ Also index the "EventTimestamp" column on the AuditTrail list separately (repeat steps 1–4 selecting EventTimestamp). This allows the Audit Trail screen to filter by date range efficiently.

Chapter 4 — Power Automate Flows

4.1 Overview

You will build **eight Power Automate flows**. Each flow:

- **Receives parameters** from the Canvas App via a "PowerApps V2" trigger
- **Writes to the primary SharePoint list** (Instruments, Consumables, or Faults)
- **Retries the audit trail write** up to 3 times if it fails
- **Returns a result text** to the Canvas App: "success", "audit_failed", or "primary_failed"

Naming convention: All flows are named **RoboticTracker - [Action]**. This keeps them grouped together in your Power Automate dashboard.

Summary of all 8 flows:

Flow name	Primary list written	Triggered when
RoboticTracker - Log Instrument Use	Instruments (update)	Nurse logs a use for a case
RoboticTracker - Log Consumable Use	Consumables (update)	Nurse logs consumable use
RoboticTracker - Log Fault	Faults (create)	Fault reported on an instrument
RoboticTracker - Undo Use	Instruments (update)	Use is reversed / undone
RoboticTracker - Change Status	Instruments (update)	Status changed (e.g. to SB Instrument)
RoboticTracker - Condemn Instrument	Instruments (update)	Instrument retired/condemned
RoboticTracker - Add Instrument	Instruments (create)	New instrument added to stock
RoboticTracker - Add Consumable	Consumables (create)	New consumable added to stock

✓ *Build Flow 4.2 first and test it completely before building the others. Flows 4.3–4.9 follow the same pattern — you will be faster after the first one.*

4.2 Flow: Log Instrument Use

This is the most important flow. Every other flow follows the same pattern with different parameters. Read this section carefully.

Create the flow

1. Go to **make.powerautomate.com** and sign in.
2. Click **Create** in the left sidebar.
3. Click **Instant cloud flow**.
4. In the "Flow name" field, type: **RoboticTracker - Log Instrument Use**
5. In the trigger list below, select **PowerApps (V2)**. Click **Create**.

Step 1: Define the trigger inputs

The "PowerApps (V2)" trigger appears as the first block. You need to add input parameters — these are the values the Canvas App will send to the flow.

6. Click the trigger block to expand it.
7. Click **+ Add an input**.
8. Add the following inputs one by one (click **+ Add an input** for each):

Input name (type exactly)	Type
serialNumber	Text
caseNumber	Text
loggedBy	Text
timestamp	Text
newUsesLeft	Number
newLastUsed	Text
newLastCase	Text
newStatus	Text

Step 2: Initialize variables

9. Click **+ New step**. Search for **Initialize variable** and select it.
10. Fill in: Name = **AuditOK**, Type = **Boolean**, Value = **false**.
11. Click **+ New step** again. Add another **Initialize variable**:
Name = **RetryCount**, Type = **Integer**, Value = **0**.

Step 3: Find the instrument in SharePoint

12. Click **+ New step**. Search for **Get items** and select **SharePoint - Get items**.
13. Fill in:
 - Site Address: select your Robotic OT Tracker site from the dropdown
 - List Name: **Instruments**
 - Filter Query: **Title eq ' (then click **Add dynamic content**, select **serialNumber**, then close the**

quote with ')

■ The Filter Query should look like: `Title eq '@{triggerBody()}'` Power Automate builds this automatically when you insert the dynamic content.

14. Expand **Show advanced options**. Set **Top Count** to **1**.

Step 4: Update the instrument

15. Click + **New step**. Search for **Update item** and select **SharePoint - Update item**.

16. Fill in:

- Site Address: your Robotic OT Tracker site
- List Name: **Instruments**
- Id: click **Add dynamic content** > **Get items** section > **ID**
(If it asks you to add a For Each loop, click Add a "For each" — this is normal because Get items can return multiple results)
- Title (Serial Number): insert dynamic content **serialNumber**
- UsesLeft: insert dynamic content **newUsesLeft**
- LastUsed: insert dynamic content **newLastUsed**
- LastCase: insert dynamic content **newLastCase**
- Status Value: insert dynamic content **newStatus**

Step 5: Retry loop for audit trail

17. Click + **New step**. Search for **Do until** and select it.

18. In the "Do Until" block, set the exit condition:

- Click the first box, select **AuditOK** from dynamic content
- Middle dropdown: select **is equal to**
- Last box: type **true**
- Click **Change limits** and set **Count** to **3**

19. Inside the Do Until loop, click **Add an action**. Search for **Create item** and select **SharePoint - Create item**.

20. Fill in the Create item (AuditTrail) action:

- Site Address: your site
- List Name: **AuditTrail**
- Title (SerialNumber): dynamic content **serialNumber**
- EventTimestamp: dynamic content **timestamp**
- Event Value: **Use logged** (type this as text)
- Detail: create an expression combining the values — type:
Uses left after: + dynamic content **newUsesLeft** + , **Case:** + **caseNumber**
- CaseNumber: dynamic content **caseNumber**
- LoggedBy: dynamic content **loggedBy**

21. Inside the loop, after the Create item action, click **Add an action**. Add a **Condition** action.

22. In the Condition:

- Left side: click "Choose a value", go to **Expression** tab, type: **outputs('Create_item')['statusCode']**, click OK
- Middle: **is equal to**
- Right side: **201** (HTTP 201 = Created successfully)

23. In the Yes branch of the Condition: click Add an action, add Set variable. Name = AuditOK, Value = true.**24. In the No branch: add Increment variable. Name = RetryCount, Value = 1. Then add a Delay action: Interval = 2, Unit = Second.**

Step 6: Respond to the Canvas App

25. After the Do Until loop (outside it), click + New step. Search for Respond to a PowerApp and select it.**26. Click + Add an output. Select Text.****27. Name the output: result.****28. For the Value, click the Expression tab and type: if(variables('AuditOK'), 'success', 'audit_failed')****29. Click Save (top right). The flow is now ready.**

Test the flow

30. Click Test (top right) > Manually > Test.**31. Enter test values (use a real serial number from your Instruments list) and click Run flow.****32. If the flow succeeds, check your SharePoint Instruments list to confirm the item was updated, and check the AuditTrail list for a new entry.**

■ If the flow fails at "Get items", check that the site address and list name are correct. If it fails at "Update item", verify the column names match exactly (case-sensitive) what you created in Chapter 3.

4.3 Flow: Log Consumable Use

Follow the same steps as Section 4.2 with these differences:

Flow name: RoboticTracker - Log Consumable Use

Input name	Type
serialNumber	Text
caseNumber	Text
loggedBy	Text
timestamp	Text

Input name	Type
newBalance	Number
newLastUsed	Text

SharePoint action differences:

- Primary action: **SharePoint - Update item** on list **Consumables**
Update fields: Title (serialNumber), Balance (newBalance), LastUsed (newLastUsed)
- AuditTrail Event value: **Consumable use logged**
- AuditTrail Detail: "Balance after: " + newBalance + ", Case: " + caseNumber

4.4 Flow: Log Fault

Flow name: RoboticTracker - Log Fault

This flow **creates** a new Faults item (not update).

Input name	Type
serialNumber	Text
instrumentType	Text
caseNumber	Text
faultType	Text
notes	Text
loggedBy	Text
timestamp	Text

Note: No "Get items" step needed. Primary action is **SharePoint - Create item** on list **Faults**. Fill in all fields from the trigger inputs. Set FaultTimestamp = timestamp input. AuditTrail Event = **Fault logged**.

4.5 Flow: Undo Use

Flow name: RoboticTracker - Undo Use

Input name	Type
serialNumber	Text
loggedBy	Text
timestamp	Text
restoredUsesLeft	Number
restoredLastUsed	Text
restoredLastCase	Text
restoredStatus	Text

Input name	Type
undoReason	Text

Same pattern as 4.2: Get items on Instruments, Update item (restore UsesLeft, LastUsed, LastCase, Status to the values passed in). AuditTrail Event = **Use REVERSED**. Detail = "Reason: " + undoReason.

4.6 Flow: Change Instrument Status

Flow name: RoboticTracker - Change Status

Input name	Type
serialNumber	Text
newStatus	Text
loggedBy	Text
timestamp	Text

Update item: update only the Status field on the Instruments list. AuditTrail Event = **Status changed**. Detail = "New status: " + newStatus.

4.7 Flow: Condemn Instrument

Flow name: RoboticTracker - Condemn Instrument

Input name	Type
serialNumber	Text
condemnReason	Text
loggedBy	Text
timestamp	Text

Update item: set Status = **Condemned**. AuditTrail Event = **Instrument condemned**. Detail = "Reason: " + condemnReason.

4.8 Flow: Add Instrument

Flow name: RoboticTracker - Add Instrument

Primary action: **Create item** on Instruments (no Get items step needed).

Input name	Type
serialNumber	Text
instrumentType	Text
status	Text

Input name	Type
maxLife	Number
usesLeft	Number
remarks	Text
loggedBy	Text
timestamp	Text

Create item fields: Title=serialNumber, InstrumentType=instrumentType, Status=status, UsesLeft=usesLeft, MaxLife=maxLife, Remarks=remarks. AuditTrail Event = **Instrument added**.

4.9 Flow: Add Consumable

Flow name: RoboticTracker - Add Consumable

Primary action: **Create item** on Consumables.

Input name	Type
serialNumber	Text
consumableType	Text
maxBalance	Number
expiry	Text
loggedBy	Text
timestamp	Text

Create item fields: Title=serialNumber, ConsumableType=consumableType, Balance=maxBalance (starts full), MaxBalance=maxBalance, Expiry=expiry. AuditTrail Event = **Consumable added**.

Chapter 5 — Build the Canvas App

5.1 Create the app and connect data

1. Go to **make.powerapps.com**. Make sure you are in the correct environment (top-right corner shows your organisation name).
2. Click **+ Create** in the left sidebar.
3. Click **Blank app**.
4. Choose **Blank canvas app**. App name: **Robotic OT Tracker**. Format: **Phone** (portrait layout, works on phones and tablets). Click **Create**.
5. The PowerApps Studio (editor) opens. You see a blank white screen in the centre, a left panel (Tree View / Insert), and a formula bar at the top.
6. Connect to SharePoint: click **Data** (cylinder icon) in the left sidebar. Click **+ Add data**. Search for **SharePoint**. Select it.
7. In the connection panel, paste your SharePoint site URL (from Chapter 3). Press Enter. You will see a list of lists on that site.
8. Select all four lists: **Instruments**, **Consumables**, **Faults**, **AuditTrail**. Click **Connect**.

■ If SharePoint is not listed as a data source, click "New connection", search for "SharePoint", and sign in with your M365 account.

Connect the 8 Power Automate flows

9. Click the **lightning bolt** icon in the left sidebar (Power Automate).
10. Click **+ Add flow**.
11. You will see your flows listed. Select all 8 flows you created in Chapter 4. Click **Add**.

■ Flows are referenced in formulas by their names. PowerApps replaces hyphens and spaces with underscores, so "RoboticTracker - Log Instrument Use" becomes "RoboticTracker_LogInstrumentUse" in your formulas.

5.2 App.OnStart — load data

The App.OnStart property runs once when the app loads. Put it here to load all data into local Collections.

1. Click on an empty area of any screen (not a control).
2. In the top-left dropdown (where it says "Screen"), change it to **App**.
3. In the property dropdown next to it (says "Fill" or similar), select **OnStart**.

4. In the formula bar, paste the following formula:

```
// Load all data from SharePoint into local collections
ClearCollect(colInstruments, Instruments);
ClearCollect(colConsumables, Consumables);
ClearCollect(colFaults, Faults);
// Load only last 90 days of audit trail (delegation-safe)
ClearCollect(colAudit,
Filter(AuditTrail,
EventTimestamp >= DateAdd(Today(), -90, TimeUnit.Days)
)
);
Set(varCurrentUser, User().DisplayName);
Set(varLoading, false);
```

Now create the screens. In the left Tree View panel, you will see "Screen1". You need to create 7 screens total. Right-click "Screen1" > Rename > type **scrDashboard**. Then add 6 more screens:

5. Click **+ New screen** (in Tree View, bottom). Choose **Blank**. Repeat 6 times. Rename each screen:
scrDashboard, scrInstruments, scrConsumables, scrLogUse, scrManage, scrFaults, scrAudit

5.3 Screen: Dashboard

The dashboard shows summary stat cards. Build it as follows:

Background and title

1. Select **scrDashboard** in the Tree View.
2. In the right Properties panel, set Fill to a teal colour: click the fill box, select Custom, enter hex **#00606E**.
3. Insert > Text label. Set Text = **"Robotic OT Tracker"**. Font size 22, bold, white colour. Position it near the top.
4. Insert > Text label. Text = **"CGH · da Vinci Programme"**. Font size 12, light teal colour.

Stat cards

Add a white rounded rectangle for each card. Inside each rectangle, add two labels: one for the number (large, bold) and one for the description.

The **Text** formula for each number label:

Card	Formula for the number label
In Circulation	CountRows(Filter(colInstruments, Status = "Circulation"))
Critical (≤2 uses)	CountRows(Filter(colInstruments, UsesLeft <= 2 And Status <> "Condemned" And Status <> "Complete"))
SB Instruments	CountRows(Filter(colInstruments, Status = "SB Instrument"))
Faults (30 days)	CountRows(Filter(colFaults, FaultTimestamp >= DateAdd(Today(), -30, TimeUnit.Days)))

Card	Formula for the number label
Consumables Available	CountRows(Filter(colConsumables, Balance > 0 And (IsBlank(Expiry) Or Expiry >= Today())))

Refresh button

5. Insert > Button. Text = "Refresh". OnSelect formula:

```
ClearCollect(colInstruments, Instruments);
ClearCollect(colConsumables, Consumables);
ClearCollect(colFaults, Faults);
ClearCollect(colAudit,
Filter(AuditTrail, EventTimestamp >= DateAdd(Today(), -90, TimeUnit.Days))
);
```

5.4 Screen: Instruments

Shows a searchable list of all instruments with status badges.

1. Insert > **Vertical gallery**. Set **Items** property to:

```
Sort(
Filter(colInstruments,
IsBlank(txtInstSearch.Text) Or
Title in txtInstSearch.Text Or
InstrumentType in txtInstSearch.Text
),
Title, SortOrder.Ascending
)
```

2. Inside the gallery, add labels for: **ThisItem.Title** (serial number), **ThisItem.InstrumentType**, **ThisItem.Status** (as a badge).

3. For the **UsesLeft progress bar**, insert a Rectangle. Set its Width formula to:

```
If(IsBlank(ThisItem.MaxLife) Or ThisItem.MaxLife = 0, 0,
Parent.Width * (ThisItem.UsesLeft / ThisItem.MaxLife)
)
```

4. For the status badge colour, insert a Rectangle behind the status label. Set its Fill to:

```
Switch(ThisItem.Status,
"Circulation", ColorValue("#2A7A2A"),
"SB Instrument", ColorValue("#1A5A9A"),
"URO Set", ColorValue("#7A2A9A"),
"Non-sterile", ColorValue("#C87A00"),
"Complete", ColorValue("#4A5568"),
"Condemned", ColorValue("#C82020"),
ColorValue("#4A5568")
)
```

5. Insert a **Text input** above the gallery. Name it **txtInstSearch**. Hint text = "Search serial number or type..."

5.5 Screen: Consumables

Same pattern as the Instruments screen.

1. Add a Vertical gallery. Items:

```
Sort(
  Filter(colConsumables,
    IsBlank(txtConsSearch.Text) Or
    Title in txtConsSearch.Text Or
    ConsumableType in txtConsSearch.Text
  ),
  Title, SortOrder.Ascending
)
```

2. Show: **ThisItem.Title**, **ThisItem.ConsumableType**, **ThisItem.Balance / ThisItem.MaxBalance**, **ThisItem.Expiry**.

3. Expiry warning — set label colour:

```
If(Not IsBlank(ThisItem.Expiry) And ThisItem.Expiry < Today(),
  Red,
  If(Not IsBlank(ThisItem.Expiry) And
    ThisItem.Expiry < DateAdd(Today(), 30, TimeUnit.Days),
    ColorValue("#C87A00"),
    RGBA(30, 30, 30, 1)
  )
)
```

5.6 Screen: Log Use

This screen handles the most critical action in the app. It lets the nurse log instrument and consumable use for a case number.

Controls to add

Control	Name	Purpose
Text input	txtCaseNum	Case visiting number (e.g. XXX456789A)
Dropdown	drpLogType	Instrument or Consumable (two options)
Gallery (vertical)	galLogItems	Shows filtered instruments or consumables
Button	btnLogUse	"Log Use" — submits the entry
Text label	lblLogStatus	Shows success/error message

Gallery Items formula

```
If(drpLogType.Selected.Value = "Instrument",  
  Filter(colInstruments,  
    Status <> "Condemned" And Status <> "Complete"),  
  Filter(colConsumables,  
    Balance > 0)  
)
```

"Log Use" button — OnSelect formula

This formula has three logical parts. Paste all three into the OnSelect property of btnLogUse, one after the other separated by semicolons.

Part 1 — Validate inputs and same-case lock

```
// Step 1: validate  
If(  
  IsBlank(galLogItems.Selected) Or IsBlank(txtCaseNum.Text),  
  Notify("Select an item and enter a case number.", NotificationType.Warning),  
  If(drpLogType.Selected.Value = "Instrument" And  
    galLogItems.Selected.LastCase = txtCaseNum.Text,  
    Notify("Already logged for case " & txtCaseNum.Text,  
      NotificationType.Warning),  
    // -- continue with Step 2 inside this If (see Part 2) --
```

Part 2 — Call the flow (inside the outer If from Part 1)


```
// Step 2: call the correct flow
Set(varProcessing, true);
If(drpLogType.Selected.Value = "Instrument",
IfError(
Set(varFlowResult,
RoboticTracker_LogInstrumentUse.Run(
galLogItems.Selected.Title, // serial number
txtCaseNum.Text, // case number
varCurrentUser, // logged by
Text(Now(), "yyyy-mm-ddThh:mm:ss"), // timestamp
galLogItems.Selected.UsesLeft - 1, // new uses left
Text(Today(), "yyyy-mm-dd"), // new last used date
txtCaseNum.Text, // new last case
If(galLogItems.Selected.UsesLeft - 1 = 0,
"Complete", // auto-condemn at 0
galLogItems.Selected.Status)
)
),
Set(varFlowResult, {result: "primary_failed"})
),
// Consumable path
IfError(
Set(varFlowResult,
RoboticTracker_LogConsumableUse.Run(
galLogItems.Selected.Title,
txtCaseNum.Text,
varCurrentUser,
Text(Now(), "yyyy-mm-ddThh:mm:ss"),
galLogItems.Selected.Balance - 1,
Text(Today(), "yyyy-mm-dd")
)
),
Set(varFlowResult, {result: "primary_failed"})
);
Set(varProcessing, false);
```

Part 3 — Handle the flow result

```
// Step 3: handle result
Switch(varFlowResult.result,
"success",
If(drpLogType.Selected.Value = "Instrument",
Patch(colInstruments, galLogItems.Selected, {
UsesLeft: galLogItems.Selected.UsesLeft - 1,
LastUsed: Text(Today(), "yyyy-mm-dd"),
LastCase: txtCaseNum.Text,
Status: If(galLogItems.Selected.UsesLeft-1=0,
"Complete",galLogItems.Selected.Status)
}),
Patch(colConsumables, galLogItems.Selected, {
Balance: galLogItems.Selected.Balance - 1,
LastUsed: Text(Today(), "yyyy-mm-dd")
})
);
Notify("Use logged successfully.", NotificationType.Success),
"audit_failed",
If(drpLogType.Selected.Value = "Instrument",
Patch(colInstruments, galLogItems.Selected, {
UsesLeft: galLogItems.Selected.UsesLeft - 1,
LastUsed: Text(Today(), "yyyy-mm-dd"),
LastCase: txtCaseNum.Text
})
);
Notify("WARNING: Use saved – AUDIT TRAIL FAILED." &
" Record manually: SN " & galLogItems.Selected.Title &
" in case " & txtCaseNum.Text,
NotificationType.Warning),
Notify("Save failed. Check connection.", NotificationType.Error)
)
) // closes same-case lock If
) // closes input validation If
```

5.7 Screen: Manage (Change Status & Condemn)

Two panels on this screen: one for changing an instrument's status, and one for condemning/retiring an instrument.

Change Status panel

Control	Name / formula
Gallery (instruments)	Items: Filter(colInstruments, Status <> "Condemned" And Status <> "Complete")
Dropdown (new status)	Items: ["Circulation","SB Instrument","URO Set","Non-sterile","Complete"]
Button "Change Status"	See formula below

```
// Change Status button OnSelect
IfError(
Set(varResult,
RoboticTracker_ChangeStatus.Run(
galManageInst.Selected.Title,
drpNewStatus.Selected.Value,
varCurrentUser,
Text(Now(), "yyyy-mm-ddThh:mm:ss")
)
),
Set(varResult, {result: "primary_failed"});
Switch(varResult.result,
"success",
Patch(colInstruments, galManageInst.Selected,
{Status: drpNewStatus.Selected.Value});
Notify("Status updated.", NotificationType.Success),
"audit_failed",
Patch(colInstruments, galManageInst.Selected,
{Status: drpNewStatus.Selected.Value});
Notify("Status updated but audit trail write failed. Record manually.",
NotificationType.Warning),
Notify("Save failed.", NotificationType.Error)
)
)
```

Condemn Instrument panel

Add a gallery of active instruments, a dropdown for the condemn reason (choices: End of life — max uses reached / Intra-op failure / Physical damage / Call back by vendor / Other), and a button:

```
// Condemn button OnSelect
IfError(
Set(varResult,
RoboticTracker_CondemnInstrument.Run(
galCondemnInst.Selected.Title,
drpCondemnReason.Selected.Value,
varCurrentUser,
Text(Now(), "yyyy-mm-ddThh:mm:ss")
)
),
Set(varResult, {result: "primary_failed"});
Switch(varResult.result,
"success",
Patch(colInstruments, galCondemnInst.Selected, {Status: "Condemned"});
Notify("Instrument condemned.", NotificationType.Success),
"audit_failed",
Patch(colInstruments, galCondemnInst.Selected, {Status: "Condemned"});
Notify("Condemned but audit trail failed. Record manually.",
NotificationType.Warning),
Notify("Save failed.", NotificationType.Error)
)
)
```

5.8 Screen: Faults

Two areas: a form to log a new fault, and a gallery showing recent faults.

Control	Name	Purpose
Gallery	galFaultInst	Select the instrument with the fault. Items: Filter(collInstruments, Status <> "Condemned")
Dropdown	drpFaultType	Type of fault. Items: ["Wire broken","Wire frail","Cannot grasp","Cannot close","Jaw unable to open","Intra-op failure","Call back","Cannot fire (stapler)","Display issue (lens)","Other"]
Text input	txtFaultCase	Case number
Text input	txtFaultNotes	Additional notes. Mode = Multiline
Button "Log Fault"		See formula below

```
// Log Fault button OnSelect
If(IsBlank(galFaultInst.Selected) Or IsBlank(drpFaultType.Selected.Value),
Notify("Select instrument and fault type.", NotificationType.Warning),
IfError(
Set(varResult,
RoboticTracker_LogFault.Run(
galFaultInst.Selected.Title,
galFaultInst.Selected.InstrumentType,
txtFaultCase.Text,
drpFaultType.Selected.Value,
txtFaultNotes.Text,
varCurrentUser,
Text(Now(), "yyyy-mm-ddThh:mm:ss")
)
),
Set(varResult, {result: "primary_failed"}));
Switch(varResult.result,
"success",
Collect(colFaults, {
Title: galFaultInst.Selected.Title,
FaultType: drpFaultType.Selected.Value,
FaultTimestamp: Now(),
CaseNumber: txtFaultCase.Text,
Notes: txtFaultNotes.Text,
LoggedBy: varCurrentUser
});
Notify("Fault logged.", NotificationType.Success),
"audit_failed",
Notify("Fault saved but audit trail failed. Record manually.",
NotificationType.Warning),
Notify("Save failed. Check connection.", NotificationType.Error)
)
)
```

5.9 Screen: Audit Trail

Shows the audit log with filtering by event type and date range.

1. Add a gallery. Set its **Items** to:

```
SortByColumns(
  Filter(colAudit,
    (IsBlank(drpAuditEvent.Selected.Value) Or
    Event = drpAuditEvent.Selected.Value)
    And
    (IsBlank(txtAuditSearch.Text) Or
    Title in txtAuditSearch.Text)
    And
    EventTimestamp >= dtAuditFrom.SelectedDate
    And
    EventTimestamp <= DateAdd(dtAuditTo.SelectedDate, 1, TimeUnit.Days)
  ),
  "EventTimestamp", Descending
)
```

2. Add a **Date picker** named **dtAuditFrom** (default: DateAdd(Today(), -7, TimeUnit.Days)).

3. Add a **Date picker** named **dtAuditTo** (default: Today()).

4. Add a **Dropdown** named **drpAuditEvent**. Items:

```
[",", "Use logged", "Consumable use logged", "Fault logged",
  "Use REVERSED", "Status changed", "Instrument condemned",
  "Instrument added", "Consumable added"]
```

5. Add a **Text input** named **txtAuditSearch** for serial number search.

6. Set the screen's **OnVisible** property to reload recent audit data:

```
ClearCollect(colAudit,
  Filter(AuditTrail,
    EventTimestamp >= DateAdd(dtAuditFrom.SelectedDate, 0, TimeUnit.Days)
  )
);
```

5.10 Navigation

Add a bottom navigation bar to every screen. The simplest approach is to create one set of navigation controls and paste them to each screen.

On each screen, insert a horizontal container or 5–7 icon buttons at the bottom. Set each button's **OnSelect** to:

Button label	OnSelect formula
Dashboard	Navigate(scrDashboard, ScreenTransition.None)
Instruments	Navigate(scrInstruments, ScreenTransition.None)
Consumables	Navigate(scrConsumables, ScreenTransition.None)
Log Use	Navigate(scrLogUse, ScreenTransition.None)

Button label	OnSelect formula
Manage	Navigate(scrManage, ScreenTransition.None)
Faults	Navigate(scrFaults, ScreenTransition.None)
Audit	Navigate(scrAudit, ScreenTransition.None)

✓ After building the navigation, publish the app and test it on your phone via the PowerApps mobile app (download from App Store / Google Play). The phone experience may differ from the browser preview.

Chapter 6 — Data Migration from Google Sheets

6.1 Export instruments and consumables

1. Open the existing Robotic OT Tracker Google Sheet.
2. Click the **Instruments** tab.
3. Go to **File > Download > Microsoft Excel (.xlsx)**. Save as **instruments_export.xlsx**.
4. Repeat for the **Consumables** tab: save as **consumables_export.xlsx**.

■ *You only need to migrate Instruments and Consumables. Faults and AuditTrail are historical records — decide with your team whether to migrate those or start fresh.*

6.2 Format the spreadsheet for SharePoint

SharePoint import requires column headers that match the list column names exactly. Open each file and rename the columns to match the SharePoint list columns.

Instruments file — required columns in the header row:

Google Sheets column	Rename to (for SharePoint)
SN / Serial	Title
Type	InstrumentType
Status	Status
Uses Left	UsesLeft
Max Life	MaxLife
Last Used	LastUsed
Last Case	LastCase
Remarks	Remarks

Consumables file — required columns:

Google Sheets column	Rename to
SN / Serial	Title
Type	ConsumableType
Balance	Balance
Max Balance	MaxBalance

Google Sheets column	Rename to
Expiry	Expiry
Last Used	LastUsed

■ **Status column values must exactly match the Choice options you set in Chapter 3 (e.g. "Circulation", "SB Instrument"). Fix any differences before importing.**

6.3 Import to SharePoint

1. Go to your **Instruments** list in SharePoint.
2. Click the grid/edit icon to enter **Quick Edit** mode (it looks like a small spreadsheet grid near the top of the list).

✓ *Quick Edit (grid view) allows you to paste directly from Excel. This is the easiest import method.*

3. In Excel, select all data rows (not the header). Copy with Ctrl+C.
4. In SharePoint Quick Edit, click the first empty cell in the Title column. Paste with Ctrl+V.
5. Review the pasted data for any errors (red cells indicate a problem, usually a Choice value that does not match).
6. Click **Exit grid view** or **Stop editing this list** to save.
7. Repeat for the Consumables list.

6.4 Verify the imported data

1. Open the Canvas App you built in Chapter 5.
2. Navigate to the Instruments screen. Count the rows — it should match the number of instruments in Google Sheets.
3. Check 5–10 random instruments for correct serial number, type, status, and uses left.
4. Navigate to Consumables. Verify balances and expiry dates.
5. Open the Instruments list in SharePoint and verify that all indexed columns are populated (no blanks in SerialNumber).

■ **Do not decommission the Google Sheets app until you have run the new PowerApps app in parallel for at least 2 weeks without issues.**

Chapter 7 — Testing Checklist

Complete all items in this checklist before going live. For each item, test it on a real M365 device (phone or tablet), not just the browser preview in PowerApps Studio.

#	Test	Expected result	Done
1	Open app, check Dashboard loads	All 5 stat cards show correct counts	■
2	Log Use: select instrument, enter valid case number, tap Log Use	Success notification; instrument UsesLeft decreases by 1; AuditTrail entry created	■
3	Log Use: try same case number twice for same instrument	Warning: "Already logged for this case"	■
4	Log Use: leave case number blank, tap Log Use	Warning: "Select an item and enter a case number"	■
5	Log Consumable: select consumable, enter case, tap Log Use	Balance decreases by 1; AuditTrail entry created	■
6	Instruments screen: type a serial number in the search box	Gallery filters to matching instrument(s)	■
7	Manage: change status of an instrument	Status badge updates; AuditTrail entry created	■
8	Manage: condemn an instrument	Status changes to Condemned; instrument no longer appears in Log Use list	■
9	Faults: log a fault with notes	Fault appears in fault gallery; AuditTrail entry created	■
10	Audit Trail: filter by event type "Use logged"	Only "Use logged" entries shown	■
11	Audit Trail: change date range	Gallery updates to show entries in selected range	■
12	Turn off Wi-Fi/mobile data and try to Log Use	Error notification: "Save failed. Check your connection"	■
13	After Wi-Fi restored, try to Log Use again	Succeeds normally; no phantom duplicate entries created	■
14	Log Use for an instrument with 1 use remaining	UsesLeft becomes 0; Status changes to Complete; no longer appears in Log Use list	■
15	Open app simultaneously on two devices; log use on one device; check the other	Second device shows updated data after tapping Refresh	■

Chapter 8 — Troubleshooting

Symptom	Likely cause	Fix
Flow returns "primary_failed" every time	SharePoint site URL incorrect, or list name misspelt in the flow	Open the flow in Power Automate. Check that the Site Address and List Name in each SharePoint action exactly match what you created in Chapter 3.
Flow runs but SharePoint item is not updated	"Get items" returned 0 results (serial number not found)	Check the Filter Query in the Get Items step. Verify the serial number exists in SharePoint (check exact case, no leading/trailing spaces).
Canvas App formula bar shows red underline on flow name	Flow was not added to the app, or was added but not connected	Go to Power Automate (lightning bolt icon) in the Canvas App. Remove the flow and re-add it.
"Delegation warning" yellow triangle on gallery Items formula	The filter column is not indexed or the formula is not delegation-safe	Ensure SerialNumber (Title) is indexed in SharePoint (Chapter 3.6). Use only supported delegation operators (=, <>, <, >, >=, <=, StartsWith) with indexed columns.
App shows "Something went wrong" on startup	SharePoint connection expired or data sources disconnected	In PowerApps Studio, go to Data panel. Remove and re-add the SharePoint connection.
Audit trail entries missing for some actions	Flow returned "audit_failed" but Canvas App did not show the warning	Check that your OnSelect formula handles the "audit_failed" case with a Notify call (not just the "success" case).
Consumable balance goes negative	No minimum-value check before calling the flow	Add a check in the Log Use OnSelect: If(galLogItems.Selected.Balance <= 0, Notify("No stock remaining", ...), ...) before calling the flow.
"Respond to a PowerApp" step is missing from the flow	Step was not added, or was deleted	Re-open the flow in Power Automate. Add "Respond to a PowerApp or flow" as the last step outside the Do Until loop. Define one Text output called "result".

Appendix A — SharePoint Column Quick Reference

Use this as a checklist when creating or verifying your SharePoint lists.

Instruments

Column display name	Internal name (used in Power Automate)	Type	Required
Serial Number	Title	Single line of text	Yes (built-in)
InstrumentType	InstrumentType	Choice	No
Status	Status	Choice	No
UsesLeft	UsesLeft	Number	No
MaxLife	MaxLife	Number	No
LastUsed	LastUsed	Date and Time	No
LastCase	LastCase	Single line of text	No
Remarks	Remarks	Multiple lines of text	No

Consumables

Column display name	Internal name	Type	Required
Serial Number	Title	Single line of text	Yes (built-in)
ConsumableType	ConsumableType	Choice	No
Balance	Balance	Number	No
MaxBalance	MaxBalance	Number	No
Expiry	Expiry	Date and Time	No
LastUsed	LastUsed	Date and Time	No

Faults

Column display name	Internal name	Type	Required
Serial Number	Title	Single line of text	Yes (built-in)
FaultTimestamp	FaultTimestamp	Date and Time	No
InstrumentType	InstrumentType	Single line of text	No

Column display name	Internal name	Type	Required
FaultType	FaultType	Choice	No
CaseNumber	CaseNumber	Single line of text	No
Notes	Notes	Multiple lines of text	No
LoggedBy	LoggedBy	Single line of text	No

AuditTrail

Column display name	Internal name	Type	Required
Serial Number	Title	Single line of text	Yes (built-in)
EventTimestamp	EventTimestamp	Date and Time	No
Event	Event	Choice	No
InstrumentType	InstrumentType	Single line of text	No
Detail	Detail	Multiple lines of text	No
CaseNumber	CaseNumber	Single line of text	No
LoggedBy	LoggedBy	Single line of text	No

Appendix B — Key Power Fx Formulas Quick Reference

App.OnStart — load all data

```
ClearCollect(colInstruments, Instruments);
ClearCollect(colConsumables, Consumables);
ClearCollect(colFaults, Faults);
ClearCollect(colAudit, Filter(AuditTrail,
EventTimestamp >= DateAdd(Today(), -90, TimeUnit.Days)));
Set(varCurrentUser, User().DisplayName);
```

Dashboard stat card formulas

Card	Formula
In Circulation	CountRows(Filter(colInstruments, Status = "Circulation"))
Critical (≤ 2 uses)	CountRows(Filter(colInstruments, UsesLeft <= 2 And Status <> "Condemned" And Status <> "Complete"))
SB Instruments	CountRows(Filter(colInstruments, Status = "SB Instrument"))
Faults (30 days)	CountRows(Filter(colFaults, FaultTimestamp >= DateAdd(Today(), -30, TimeUnit.Days)))
Consumables Available	CountRows(Filter(colConsumables, Balance > 0 And (IsBlank(Expiry) Or Expiry >= Today())))

Flow call pattern (all write operations)

```
IfError(
Set(varResult, FlowName.Run(param1, param2, ...)),
Set(varResult, {result: "primary_failed"}))
);
Switch(varResult.result,
"success", /* update local collection + success notify */,
"audit_failed", /* update local collection + WARNING notify */,
/* default */ /* error notify – nothing was saved */
)
```

Status badge colour

```
Switch(ThisItem.Status,
"Circulation", ColorValue("#2A7A2A"),
"SB Instrument", ColorValue("#1A5A9A"),
"URO Set", ColorValue("#7A2A9A"),
"Non-sterile", ColorValue("#C87A00"),
"Complete", ColorValue("#4A5568"),
"Condemned", ColorValue("#C82020"),
ColorValue("#4A5568")
)
```

Instruments gallery — UsesLeft progress bar width

```
If(IsBlank(ThisItem.MaxLife) Or ThisItem.MaxLife = 0, 0,
Parent.Width * (ThisItem.UsesLeft / ThisItem.MaxLife)
)
```

Consumables expiry warning colour

```
If(Not IsBlank(ThisItem.Expiry) And ThisItem.Expiry < Today(),
Red,
If(Not IsBlank(ThisItem.Expiry) And
ThisItem.Expiry < DateAdd(Today(), 30, TimeUnit.Days),
ColorValue("#C87A00"),
RGBA(30, 30, 30, 1)
)
)
```

Navigation button

```
Navigate(scrDashboard, ScreenTransition.None)
```

Reload audit trail for a selected date range

```
ClearCollect(colAudit,
Filter(AuditTrail,
EventTimestamp >= dtAuditFrom.SelectedDate And
EventTimestamp <= DateAdd(dtAuditTo.SelectedDate, 1, TimeUnit.Days)
)
);
```

SureForm reload section (conditional visibility)

On the Log Use screen, if you need to show a special section for SureForm consumables (which use reload colours), set the Visible property of the SureForm section to:

```
drpLogType.Selected.Value = "Consumable" And
Lower(galLogItems.Selected.ConsumableType) in "sureform"
```